CHAPTER 1

The Transformer architecture

Chapters **??** through **??** developed the foundations of neural representation learning, recurrent sequence transitions, statistical language modeling, and dense vector embeddings. Through these developments, language models acquired the mathematical capacity to represent discrete tokens in continuous semantic vector spaces and to optimize parameterized neural transitions using gradient backpropagation.

Despite these breakthroughs, sequence-to-sequence modeling remained constrained by the architectural paradigm established by recurrent networks. Recurrent sequence modeling relies on the fundamental transition step $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$, which enforces an unbroken, strictly sequential computational chain. This dependency imposes three prohibitive operational bottlenecks:

1. **The sequential execution barrier**: Computing $\mathbf{h}_t$ strictly requires the prior memory write of $\mathbf{h}_{t-1}$. Consequently, an input sequence of length L demands $O(L)$ serialized operational steps, preventing modern hardware accelerators (GPUs and TPUs) from exploiting parallel tensor arithmetic across the temporal dimension.

2. **The fixed-vector information compression bottleneck**: In classical sequence-to-sequence encoder-decoder networks, an encoder compresses an input sequence of arbitrary length L into a single, fixed-size latent vector $\mathbf{h}_L \in \mathbb{R}^{d_h}$. As sequence length expands across hundreds or thousands of tokens, compressing intricate semantic, syntactic, and structural dependencies into a finite-dimensional vector induces severe information dissipation and contextual amnesia.

3. **Long-range gradient decay**: Information and backpropagation error signals propagating between distant tokens must traverse an unbroken chain of L recurrent transitions. As proven by the recurrent Jacobian product $\prod_{k=t+1}^{L} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}$ in Chapter **??**, this path length leads to exponential gradient decay or explosion, severely limiting long-range dependency modeling.

To resolve this impasse, Vaswani et al. introduced the **Transformer** [**?**]. The Transformer completely eliminates recurrent transitions, replacing recurrence with multi-head attention mechanisms. Under this paradigm, every token in a sequence communicates directly with every other token across an $O(1)$ path length in parallel, while maintaining uncompressed sequence representations $\mathbf{H} \in \mathbb{R}^{L \times d_{\text{model}}}$.

This chapter presents a comprehensive, first-principles analysis of the Transformer architecture. Section 1.1 establishes the foundations of tokenization, comparing character-, word-, and subword-level granularities, deriving the Byte Pair Encoding (BPE) algorithm with empirical Zipfian compression dynamics, and detailing operational special tokens. Section 1.2 examines positional representations, formalizing the permutation equivariance of self-attention, developing sinusoidal and learned absolute positional encodings, and formulating Shaw et al.'s relative positional representations. Section 1.3 derives the Attention primitive from classical dictionary lookup to continuous scaled dot-product attention, complete with geometric projections, variance

scaling proofs, and an exact numerical walkthrough. Section 1.4 analyzes the three fundamental attention types: bidirectional encoder self-attention, causal masked decoder self-attention, and cross-attention. Finally, Section 1.5 dissects Multi-Head Attention, Feed-Forward Networks, Residual Connections, and Layer Normalization, culminating in an end-to-end translation journey tracing tokens through the full architecture.

1.1 Tokenization: granularities, BPE mechanics, and special tokens

1.1.1 Why tokenization is mandatory

Natural language text in written documents and software engineering source code is stored as continuous, unsegmented sequences of characters encoded in digital formats such as ASCII or UTF-8. Conversely, modern neural network architectures and language models operate exclusively on dense, continuous numerical tensors $\mathbf{x} \in \mathbb{R}^d$ through multivariable matrix operations and gradient-based backpropagation.

Tokenization establishes the fundamental discrete segmentation function $\mathcal{T} : \mathcal{S} \to \{t_1, t_2, \ldots, t_N\}$ that partitions a raw input string into discrete, atomic symbolic units known as **tokens**. Without tokenization, neural architectures have no mechanism to parse continuous character sequences into structured units amenable to vocabulary mapping and continuous vector projection.

Choosing the granular unit of tokenization represents a critical architectural trade-off that directly dictates the operational characteristics of the entire language model. Specifically, tokenization establishes a fundamental tension between **vocabulary size** $|\mathcal{V}|$ and **sequence length** N:

$$|\mathcal{V}| \uparrow \implies N \downarrow \quad \text{and} \quad |\mathcal{V}| \downarrow \implies N \uparrow . \tag{1.1}$$

Because Transformer self-attention incurs quadratic computational and memory complexity $O(N^2)$ with respect to sequence length, an inflated sequence length imposes severe hardware burdens. Conversely, an inflated vocabulary size expands the memory footprint of the token embedding matrix $\mathbf{W}_{\text{emb}} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ and the final classification head $\mathbf{W}_{\text{head}} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$, while rendering rare words severely under-trained. To navigate this landscape, natural language processing historically explored three primary tokenization paradigms: character-level, word-level, and subword-level tokenization.

1.1.2 Character-level tokenization

Character-level tokenization segments input text into individual typographic characters or single raw bytes. Under this regime, every letter, digit, whitespace symbol, and punctuation mark is treated as an isolated, atomic token.

Consider the illustrative sentence from Arthur Conan Doyle's *The Adventure of the Copper Beeches*:

```
"Mrs.  Rucastle was downstairs, so I had an admirable opportunity."
```

Under character-level tokenization, this sequence decomposes into an extensive array of 68 individual character tokens:

```
['M','r','s','.',' ','R','u','c','a','s','t','l','e',' ',...,'y','.'].
```
(1.2)

Advantages of character-level tokenization

1. **Compact vocabulary**: The base vocabulary size is strictly bounded by the alphabet of the language. For standard English ASCII text, $|\mathcal{V}| \approx 100$ characters; even across multilingual Unicode alphabets, the vocabulary rarely exceeds a few hundred entries.
2. **Zero out-of-vocabulary (OOV) failure**: Because any conceivable word is merely an arrangement of characters from the base alphabet, character-level models never encounter unknown words. Every rare technical term, neologism, or novel software variable can be ingested without falling back to an uninformative unknown token placeholder.
3. **Robustness to orthographic noise and typos**: Minor misspellings (e.g., `"recieved"` instead of `"received"`) alter only one or two characters, allowing the model to leverage morphological similarity and maintain representational stability.

Disadvantages of character-level tokenization

1. **Sequence length explosion**: Sentences expand into lengthy sequences that are 4 to 6 times longer than their word-level counterparts. In Transformer models where self-attention memory and computation scale as $O(N^2)$, quadrupling sequence length increases computational overhead by a factor of 16, rendering long-context modeling computationally prohibitive.

2. **Weak semantic signal per token**: Individual characters carry negligible semantic meaning in isolation. The letter `'e'` or `'t'` provides almost zero context regarding the semantic topic of a document. Consequently, the model must dedicate multiple neural layers merely to composing characters into words before learning higher-level syntactic and semantic relationships.

3. **Representational inefficiency for common words**: Frequent, high-utility words such as `"the"`, `"and"`, or `"is"` must be redundantly reconstructed character-by-character at every occurrence, wasting valuable attention capacity.

1.1.3 Word-level tokenization

Word-level tokenization segments text along whitespace boundaries and orthographic punctuation marks, assigning an individual token identifier to each complete, autonomous word.

Applying word-level tokenization to our running example produces a compact sequence of 11 distinct tokens:

```
["Mrs.","Rucastle","was","downstairs",",","so","I","had","an","admirable","opportunity","."].
```
(1.3)

Advantages of word-level tokenization

1. **Direct semantic correspondence**: Words are the primary lexical units of human language. Each token vector directly encapsulates semantic and syntactic attributes (e.g., entity types, verb tenses, nominal properties), enabling models to learn grammatical relationships rapidly.

2. **Compact sequence length**: Because each token represents an entire word, input sequence lengths remain short ($N \approx \frac{1}{5}N_{\text{char}}$). This property preserves the quadratic compute budget of self-attention, enabling models to process extensive documents within moderate memory footprints.

3. **Interpretability and conceptual transparency**: Attention weights and intermediate representations map directly onto human-comprehensible concepts and lexical dependencies.

Disadvantages of word-level tokenization

1. **Severe out-of-vocabulary (OOV) bottleneck**: Any word not present in the pre-defined training vocabulary $\mathcal{V}$ cannot be represented, and must be mapped to the generic unknown token `<UNK>`. In our running example, the proper noun `"Rucastle"` is an uncommon surname; in a standard corpus, it would collapse into `<UNK>`, irreversibly discarding its identity.

2. **Vocabulary size explosion**: Natural languages exhibit rich vocabularies that follow Zipf's law. Capturing comprehensive coverage across broad domains requires vocabularies spanning hundreds of thousands or millions of words ($|\mathcal{V}| \geq 500{,}000$). This inflates the

parameter count of the embedding and output projection matrices, consuming gigabytes of accelerator memory.

3. **Under-training of rare words**: In large vocabularies, the long tail of rare words appears only a handful of times across billions of training tokens. Consequently, their corresponding embedding vectors receive infrequent gradient updates and fail to converge to robust semantic representations.

4. **Morphological blindness**: Word-level tokenization treats inflections and derivations as completely unrelated entities. The tokens `"teach"`, `"teacher"`, `"teaching"`, and `"teaches"` are assigned distinct indices and independent rows in the embedding table, sharing zero statistical parameters despite their shared lexical root.

1.1.4 Subword-level tokenization

Subword-level tokenization represents an optimal hybrid paradigm that synthesizes the strengths of word-level and character-level representations while systematically eliminating their core failure modes. Under this approach, frequent words are preserved as complete single tokens, while rare words, inflected variants, and compound terms are decomposed into smaller, morphologically meaningful subword chunks or base characters.

Applying subword tokenization to our running example decomposes rare or morphologically complex terms while preserving common lexical units:

```
["Mrs.","Ru","cast","le","was","downstairs",",","so","I","had","an","admirable","opportunity"
```
(1.4)

Here, the uncommon surname `"Rucastle"` is losslessly segmented into known subwords `["Ru", "cast", "le"]`, allowing the model to process the word without collapsing into `<UNK>` and without inflating the overall sequence length.

Advantages of subword-level tokenization

1. **Optimal vocabulary-length equilibrium**: Subword tokenizers operate with manageable vocabulary sizes (typically $|\mathcal{V}| \in [32{,}000, 100{,}000]$) while maintaining sequence lengths only slightly longer than word-level tokenization.

2. **Complete resolution of the OOV barrier**: Because the base vocabulary contains all fundamental characters (or raw bytes), any novel, unseen word can always be segmented into its constituent characters as a fallback. The model achieves 100% lexical coverage without ever needing an `<UNK>` token.

3. **Morphological parameter sharing**: Subword decomposition inherently captures shared prefixes, roots, and suffixes. For example, `"unhappiness"` decomposes into `["un", "happi", "ness"]`, allowing the negation prefix `"un-"` and noun suffix `"-ness"` to share statistical evidence across thousands of words.

4. **High compute and data efficiency**: Frequent words are processed in a single step, while computational capacity is dynamically allocated to dissecting rarer, more complex lexical structures.

Disadvantages of subword-level tokenization

1. **Non-intuitive segmentations**: Data-driven statistical subword algorithms occasionally create subword boundaries that do not align with human linguistic morphology, producing non-intuitive segmentations.

2. **Pre-tokenization pipeline dependency**: Subword tokenizers must be trained empirically on large representative text corpora prior to model pretraining, and tokenization models cannot be easily modified post-training without retraining the entire language model.

Table 1.1 summarizes the trade-offs across these three tokenization regimes.

Table 1.1: Systematic comparison across tokenization paradigms.

Property	Character-Level	Word-Level	Subword-Level (BPE)
Vocabulary size	Extremely small ($\|\mathcal{V}\| \approx 10^2$)	Massive ($\|\mathcal{V}\| \geq 10^5 - 10^6$)	Balanced ($\|\mathcal{V}\| \approx 3 \times 10^4 - 10^5$)
Sequence length	Longest ($4\times - 6\times$ words)	Shortest (1 token / word)	Moderate ($1.2\times - 1.4\times$ words)
OOV vulnerability	Zero (universal coverage)	Acute (frequent `<UNK>`)	Zero (character/byte fallback)
Morphological sharing	None (too granular)	None (isolated entries)	Extensive (shared subwords)
Attention compute cost	Highest ($O(N^2_{\text{char}})$)	Lowest ($O(N^2_{\text{word}})$)	Balanced ($O(N^2_{\text{subword}})$)
Modern deployment	Specialized character models	Deprecated in modern LLMs	Standard across all modern LLMs

1.1.5 Byte Pair Encoding (BPE)

To construct an optimal subword vocabulary empirically from data without relying on handcrafted linguistic rules, modern Large Language Models deploy **Byte Pair Encoding (BPE)**. Originally invented by Philip Gage in 1994 as a statistical data compression algorithm [**?**], BPE was adapted to neural natural language processing by Sennrich, Haddow, and Birch in 2016 for neural machine translation [**?**].

Desiderata of the BPE algorithm

The fundamental design goals of BPE for natural language modeling are:

1. **Data-driven subword discovery**: Automatically identify high-utility subwords from raw corpus statistics without requiring language-specific grammatical rules or morphological dictionaries.

2. **Frequency-conditioned segmentation**: Map frequent character sequences (common words and frequent grammatical affixes) into single atomic tokens, while reserving multi-token decompositions for rare words.

3. **Explicit vocabulary budgeting**: Allow model designers to specify an exact target vocabulary budget V_{target} that satisfies memory and computational constraints.

4. **Lossless character/byte fallback**: Guarantee that any arbitrary sequence of text can be represented using base symbols without information loss.

The 5-step BPE training algorithm

The BPE vocabulary construction algorithm operates through an iterative greedy bottom-up merge procedure:

1. **Initialization**: Initialize the base vocabulary $\mathcal{V}_0$ with all individual base characters occurring across the training corpus $\mathcal{C}$. To represent whitespace losslessly, whitespace is either converted into an explicit symbol (such as '␣' or 'Ġ') or preserved directly via byte-level encoding. Every word in the training corpus is split into a contiguous sequence of individual characters.

2. **Adjacent pair frequency counting**: Scan the entire tokenized corpus and compute the frequency histogram of all contiguous adjacent token pairs (p_1, p_2):

$$\text{Freq}(p_1, p_2) = \sum_{w \in \mathcal{C}} \text{Count}(w) \cdot \mathbb{I}\{(p_1, p_2) \subseteq w\}. \tag{1.5}$$

3. **Greedy merge candidate selection**: Identify the most frequent adjacent pair (F_1, F_2) across the corpus:

$$(F_1, F_2) = \arg\max_{(p_1, p_2)} \text{Freq}(p_1, p_2). \tag{1.6}$$

   Assign this pair a new, unique merged token identifier $T = F_1 \circ F_2$, and log the merge rule $(F_1, F_2) \mapsto T$ into an ordered merge table $\mathcal{M}$.

4. **In-place corpus contraction**: Replace every adjacent occurrence of (F_1, F_2) across the entire corpus with the new symbol T, and expand the vocabulary:

$$\mathcal{V}_{k+1} = \mathcal{V}_k \cup \{T\}. \tag{1.7}$$

5. **Iterative repetition**: Repeat Steps 2 through 4 until reaching the target vocabulary budget $|\mathcal{V}| = V_{\text{target}}$ or until no pair frequency exceeds a minimum threshold.

Byte-level BPE: eliminating unknown tokens via UTF-8

In standard character-level BPE, multilingual corpora spanning non-Latin scripts (e.g., Cyrillic, Chinese, Arabic, mathematical symbols, emojis) can expand the base alphabet into thousands of unique Unicode characters. If an unseen Unicode character appears during inference, standard BPE still fails and outputs `<UNK>`.

To achieve universal lexical coverage, Radford et al. [**?**] introduced **Byte-Level BPE** for GPT-2. Byte-level BPE initializes the base alphabet not with Unicode characters, but with the **256 possible values of a raw byte** (0x00 through 0xFF). Because the UTF-8 standard encodes all 1,114,112 Unicode code points using variable-length sequences of 1 to 4 bytes, initializing with 256 byte tokens guarantees that *any digital file or textual string can be encoded without ever encountering an unknown token*:

$$\mathcal{V}_{\text{base}} = \{0\text{x}00, 0\text{x}01, \ldots, 0\text{x}FF\}, \quad |\mathcal{V}_{\text{base}}| = 256. \tag{1.8}$$

Byte-level BPE preserves ASCII characters directly as single bytes while naturally merging multi-byte UTF-8 sequences into frequent subwords during training.

Worked walkthrough: the slide example on "would a woodchuck chuck wood"

To trace the mechanical operation of BPE, consider the exact worked example presented in the lecture slides, operating on the tongue-twister corpus:

```
"would a woodchuck chuck wood"
```

Let whitespace be explicitly represented by the boundary symbol '␣'.
Step 1: Initial character encoding.
The raw corpus decomposes into 28 initial character tokens:

|w|o|u|l|d|␣|a|␣|w|o|o|d|c|h|u|c|k|␣|c|h|u|c|k|␣|w|o|o|d|

The initial base vocabulary comprises 10 unique character symbols:

$$\mathcal{V}_0 = \{\text{'␣','a','c','d','h','k','l','o','u','w'}\}, \quad |\mathcal{V}_0| = 10. \tag{1.9}$$

Step 2: Adjacent pair frequency counting.
Scanning the 28 contiguous tokens across the corpus produces the initial pair frequency histogram:

- ('w','o') : 3 (occurring in "would", "woodchuck", and "wood").
- ('␣','w') : 2, ('o','o') : 2, ('o','d') : 2.
- ('c','h') : 2, ('h','u') : 2, ('u','c') : 2, ('c','k') : 2, ('k','␣') : 2.
- All other pairs (such as ('o','u'), ('u','l'), ('l','d'), ('␣','a'), ('a','␣'), ('d','c'), ('␣','c')) appear exactly once (1).

Step 3 & 4: Merge 1.
The most frequent pair is ('w','o') with frequency 3. We create the merged token "wo" and update the vocabulary: $\mathcal{V}_1 = \mathcal{V}_0 \cup \{\text{"wo"}\}$. Replacing all occurrences of ('w','o') in the corpus yields:

|wo|u|l|d|␣|a|␣|wo|o|d|c|h|u|c|k|␣|c|h|u|c|k|␣|wo|o|d|

The sequence length contracts from 28 to 25 tokens.
Subsequent Merge Iterations.
At iteration 2, multiple pairs share the maximum frequency of 2. Following the slide progression:

- **Merge 2**: Pair ('␣',"wo") (frequency 2) merges into "␣wo". Corpus: |wo|u|l|d|␣|a|␣wo|o|d|c|h|u|c|k (23 tokens).
- **Merge 3**: Pair ("␣wo",'o') (frequency 2) merges into "␣woo". Corpus: |wo|u|l|d|␣|a|␣woo|d|c|h|u|c| (21 tokens).
- **Merge 4**: Pair ("␣woo",'d') (frequency 2) merges into "␣wood". Corpus: |wo|u|l|d|␣|a|␣wood|c|h|u|c (19 tokens).
- **Merge 5**: Pair ('c','h') (frequency 2) merges into "ch". Corpus: |wo|u|l|d|␣|a|␣wood|ch|u|c|k|␣|ch (17 tokens).
- **Merge 6**: Pair ("ch",'u') (frequency 2) merges into "chu". Corpus: |wo|u|l|d|␣|a|␣wood|chu|c|k|␣|c (15 tokens).
- **Merge 7**: Pair ("chu",'c') (frequency 2) merges into "chuc". Corpus: |wo|u|l|d|␣|a|␣wood|chuc|k|␣| (13 tokens).
- **Merge 8**: Pair ("chuc",'k') (frequency 2) merges into "chuck". Corpus: |wo|u|l|d|␣|a|␣wood|chuck|␣ (11 tokens).
- **Merge 9**: Pair ("wo",'u') merges into "wou". Corpus: |wou|l|d|␣|a|␣wood|chuck|␣|chuck|␣wood| (10 tokens).

Table 1.2 documents the progressive contraction of sequence length alongside the expansion of the subword vocabulary across these merge steps.

Table 1.2: BPE merge progression on the corpus "would a woodchuck chuck wood".

Iter.	Merged Pair	New Token	Segmented Corpus State	Tokens
0	—	—	|w|o|u|l|d|␣|a|␣|w|o|o|d|c|h|u|c|k|␣|c|h|u|c|k|␣|w|o|o|d|	28
1	('w','o')	"wo"	|wo|u|l|d|␣|a|␣|wo|o|d|c|h|u|c|k|␣|c|h|u|c|k|␣|wo|o|d|	25
2	('␣',"wo")	"␣wo"	|wo|u|l|d|␣|a|␣wo|o|d|c|h|u|c|k|␣|c|h|u|c|k|␣wo|o|d|	23
3	("␣wo",'o')	"␣woo"	|wo|u|l|d|␣|a|␣woo|d|c|h|u|c|k|␣|c|h|u|c|k|␣woo|d|	21
4	("␣woo",'d')	"␣wood"	|wo|u|l|d|␣|a|␣wood|c|h|u|c|k|␣|c|h|u|c|k|␣wood|	19
5	('c','h')	"ch"	|wo|u|l|d|␣|a|␣wood|ch|u|c|k|␣|ch|u|c|k|␣wood|	17
6	("ch",'u')	"chu"	|wo|u|l|d|␣|a|␣wood|chu|c|k|␣|chu|c|k|␣wood|	15
7	("chu",'c')	"chuc"	|wo|u|l|d|␣|a|␣wood|chuc|k|␣|chuc|k|␣wood|	13
8	("chuc",'k')	"chuck"	|wo|u|l|d|␣|a|␣wood|chuck|␣|chuck|␣wood|	11
9	("wo",'u')	"wou"	|wou|l|d|␣|a|␣wood|chuck|␣|chuck|␣wood|	10

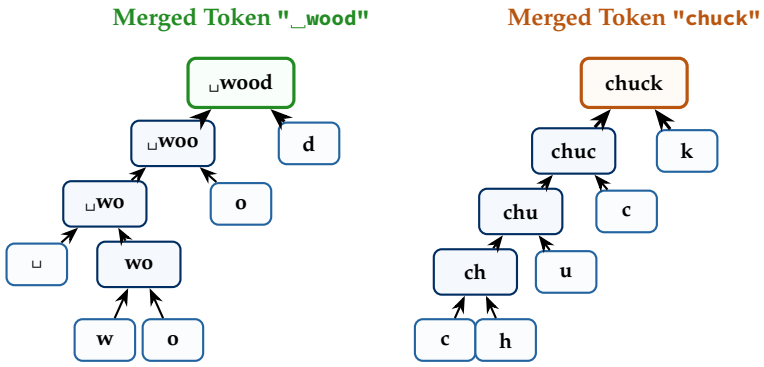


Figure 1.1: Hierarchical BPE merge tree representing the iterative composition of subwords into atomic tokens "␣wood" and "chuck" across the slide example.

Empirical compression dynamics and Zipf's law

As established empirically by Sennrich et al. [**?**] and emphasized in Slide 28, the number of tokens required to represent a corpus exhibits an exponential decay as a function of the number of merge iterations.

This behavior is a direct manifestation of **Zipf's law**: a small fraction of character combinations account for the vast majority of textual occurrences. Consequently, the first few thousand merges achieve dramatic text compression, collapsing frequent grammatical particles, affixes, and ubiquitous root words into single tokens. Beyond 30,000 to 50,000 merge operations, the marginal compression benefit flattens significantly, as subsequent merges capture increasingly rare and specialized domain words. In foundation models, vocabulary budgets typically range from 32,000 (e.g., LLaMA) to 100,000+ (e.g., GPT-4), striking an empirical balance between sequence length compression and embedding table memory overhead.

1.1.6 Special tokens in modern language models

In addition to linguistic subwords, neural sequence models require explicit structural control markers. These operational signals are designated as **special tokens**. Special tokens occupy dedicated indices in the vocabulary $\mathcal{V}$ and provide non-linguistic conditioning signals during training and autoregressive generation:

1. **<BOS> / <s> (Beginning of Sequence)**: Prepended to the input sequence to indicate the starting boundary of a text stream. In encoder-decoder and decoder-only generation, feeding <BOS> into the decoder serves as the initial prompt to trigger the generation of the very first output token when no prior context exists.

2. **<EOS> / </s> (End of Sequence)**: Appended to the end of target sentences. During autoregressive inference, generating <EOS> acts as the stopping criterion that halts generation, preventing unbounded repetition.

3. **<PAD> (Padding Token)**: When grouping sequences of variable lengths into a mini-batch tensor $\mathbf{X} \in \mathbb{R}^{B \times L_{\max}}$, shorter sequences are padded with <PAD> tokens to achieve rectangular tensor dimensions. Crucially, padding positions are masked out during self-attention so that real tokens do not attend to artificial padding.

4. **<UNK> (Unknown Token)**: Serves as a placeholder for words or characters absent from discrete vocabularies. In modern byte-level BPE models, <UNK> is rendered obsolete because any character can be decomposed into base UTF-8 bytes.

5. **<CLS> (Classification Token)**: Introduced in BERT [**?**], this token is prepended to the input sequence. Because its bidirectional attention representation pools contextual signals across all positions, the final hidden state of <CLS> is extracted as the sequence-level semantic summary for classification tasks.

6. **<SEP> (Separator Token)**: Demarcates boundaries between distinct segments within a single input (e.g., separating premise and hypothesis in natural language inference, or context and question in reading comprehension).

7. **<MASK> (Mask Token)**: Used during masked language model pretraining (e.g., BERT) to replace a subset of input tokens (typically 15%), forcing the bidirectional encoder to predict the original identity of the masked token from surrounding bidirectional context.

1.2 Positional representations: from permutation equivariance to relative encodings

1.2.1 Motivation: permutation equivariance of self-attention

In Recurrent Neural Networks, sequential order is inherently baked into the temporal transition equation: $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$. By construction, token $\mathbf{x}_t$ cannot be processed until token $\mathbf{x}_{t-1}$ has updated the hidden state. In Convolutional Neural Networks, sequential order is preserved through localized 1D sliding receptive fields.

In stark contrast, the core mathematical operator of the Transformer—**Self-Attention**—possesses zero intrinsic knowledge of sequential or temporal order. As established in Theorem 1.1, self-attention is strictly **permutation-equivariant**.

Theorem 1.1: *Permutation equivariance of self-attention*

Let $\mathbf{X} \in \mathbb{R}^{L\times d}$ be an input sequence of L token vectors, and let $\mathbf{P} \in \{0,1\}^{L\times L}$ be an arbitrary permutation matrix satisfying $\mathbf{P}^\top\mathbf{P} = \mathbf{I}$. Under standard scaled dot-product attention:

$$\texttt{Attention}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q(\mathbf{X}\mathbf{W}_K)^\top}{\sqrt{d_k}}\right)\mathbf{X}\mathbf{W}_V, \tag{1.10}$$

the operator is equivariant under input permutations:

$$\texttt{Attention}(\mathbf{P}\mathbf{X}) = \mathbf{P}\,\texttt{Attention}(\mathbf{X}). \tag{1.11}$$

Proof. Let $\mathbf{X}' = \mathbf{P}\mathbf{X}$. The linearly projected query and key matrices become $\mathbf{Q}' = \mathbf{P}\mathbf{X}\mathbf{W}_Q$ and $\mathbf{K}' = \mathbf{P}\mathbf{X}\mathbf{W}_K$. The unscaled inner-product score matrix evaluates to:

$$\mathbf{Q}'(\mathbf{K}')^\top = (\mathbf{P}\mathbf{X}\mathbf{W}_Q)(\mathbf{P}\mathbf{X}\mathbf{W}_K)^\top = \mathbf{P}(\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top)\mathbf{P}^\top. \tag{1.12}$$

Because the softmax operator evaluates independently along each row, applying a permutation matrix $\mathbf{P}$ on the left permutes the rows, while $\mathbf{P}^\top$ on the right permutes the columns:

$$\text{softmax}\left(\frac{\mathbf{P}(\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top)\mathbf{P}^\top}{\sqrt{d_k}}\right) = \mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)\mathbf{P}^\top. \tag{1.13}$$

Multiplying by the value projection $\mathbf{V}' = \mathbf{P}\mathbf{X}\mathbf{W}_V$:

$$\begin{aligned}\texttt{Attention}(\mathbf{P}\mathbf{X}) &= \left[\mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)\mathbf{P}^\top\right](\mathbf{P}\mathbf{X}\mathbf{W}_V)\\ &= \mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)(\mathbf{P}^\top\mathbf{P})\mathbf{X}\mathbf{W}_V\\ &= \mathbf{P}\,\texttt{Attention}(\mathbf{X}),\end{aligned} \tag{1.14}$$

where the identity $\mathbf{P}^\top\mathbf{P} = \mathbf{I}$ collapses the internal permutation. ■ □

Theorem 1.1 demonstrates that if the input sequence is permuted, the resulting representations undergo the exact same permutation with identical values. Self-attention operates on the sequence as an **unordered multiset** (a "bag of vectors").

As highlighted in Slide 32, this property creates an immediate syntactic dilemma. Consider two sentences composed of the exact same lexical units:

$$\begin{aligned} &\textbf{Input 1: } \textit{"I ate an apple"} \quad \Longrightarrow \quad \{\mathbf{x}_{\text{I}}, \mathbf{x}_{\text{ate}}, \mathbf{x}_{\text{an}}, \mathbf{x}_{\text{apple}}\}, \\ &\textbf{Input 2: } \textit{"an apple, I ate"} \quad \Longrightarrow \quad \{\mathbf{x}_{\text{an}}, \mathbf{x}_{\text{apple}}, \mathbf{x}_{\text{I}}, \mathbf{x}_{\text{ate}}\}. \end{aligned}$$

Without positional information, both sentences generate identical multiset representations, completely conflating grammatical subject, verb, and direct object. To enable Transformers to understand grammatical syntax, sequential order must be injected into the representations explicitly.

1.2.2 Taxonomy of positional representations

Positional representations can be categorized systematically across two independent architectural axes:

1. **Fixed vs. Learned**:
   - *Fixed Encodings*: Fully deterministic mathematical functions of position coordinates, requiring zero trainable parameters (e.g., Sinusoidal Encodings [**?**]).
   - *Learned Embeddings*: Trainable parameter matrices $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d_{\text{model}}}$ optimized via gradient descent alongside token embeddings (e.g., GPT-2 [**?**], BERT [**?**]).
2. **Absolute vs. Relative**:
   - *Absolute Positional Embeddings*: Assign an isolated vector to each discrete absolute coordinate $pos \in \{0, 1, \ldots, L-1\}$, additively combined with the token embedding prior to the first layer: $\mathbf{z}_i = \mathbf{x}_i + \mathbf{p}_i$.
   - *Relative Positional Embeddings*: Parameterize the relative distance $j - i$ between tokens, injecting positional bias directly into attention logits during attention computation (e.g., Shaw et al. [**?**], RoPE [**?**], ALiBi [**?**]).

Table 1.3 summarizes this two-dimensional classification.

Table 1.3: Taxonomy of positional representations across modern architectures.

Paradigms	Fixed (Deterministic)	Learned (Data-Driven)
Absolute	**Sinusoidal Encodings** [**?**] sin/cos harmonic geometric frequencies	**Learned Absolute Embeddings** [**? ?**] Trainable lookup table $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d_{\text{model}}}$
Relative	**Rotary Embeddings (RoPE)** [**?**] Orthogonal 2D rotation of queries/keys	**Shaw et al. Encodings** [**?**] Trainable relative bias matrix $\mathbf{S}_{\text{rel}}$ in attention

1.2.3 Absolute positional embeddings

Sinusoidal positional encodings: formulation and geometric clock intuition

In the landmark paper *Attention Is All You Need* [**?**], Vaswani et al. introduced fixed sinusoidal positional encodings. For a token at integer sequence position $pos \in \{0, 1, \ldots, L-1\}$ and feature dimension index $i \in \{0, 1, \ldots, \frac{d_{\text{model}}}{2} - 1\}$, the positional vector $\mathbf{p}_{pos} = \mathsf{PE}_{(pos,:)} \in \mathbb{R}^{d_{\text{model}}}$ is defined by:

$$\mathsf{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right), \tag{1.15}$$

$$\mathsf{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right). \tag{1.16}$$

The Geometric Clock Analogy.
To understand the geometric intuition underlying this formula, consider an analog clock with multiple hands rotating at different speeds. In a multi-dial clock:

- **Low-dimension channels ($i \to 0$)**: Oscillate at high angular frequencies $\omega_0 = 1.0$, corresponding to a rapid *seconds hand*. Wavelength $\lambda_0 = 2\pi \approx 6.28$ tokens. This rapid oscillation disambiguates immediate neighbors and local syntactic relationships.
- **High-dimension channels ($i \to \frac{d_{\text{model}}}{2} - 1$)**: Oscillate at low angular frequencies $\omega \to \frac{1}{10000}$, corresponding to a slow *hours hand*. Wavelength $\lambda_{\max} = 2\pi \cdot 10000 \approx 62{,}832$ tokens. This slow progression captures global sequence context and document flow.

Pairing sin and cos across dimension pairs $(2i, 2i+1)$ eliminates 1D wave aliasing. Because $\sin(\theta) = \sin(\pi - \theta)$, observing height along a single sine wave creates directional ambiguity (e.g., 2:00 and 10:00 produce identical vertical coordinates). By pairing sine and cosine, the vector $(\sin\theta, \cos\theta)$ traces a unique, unambiguous point along the unit circle S^1 in 2D Euclidean space.

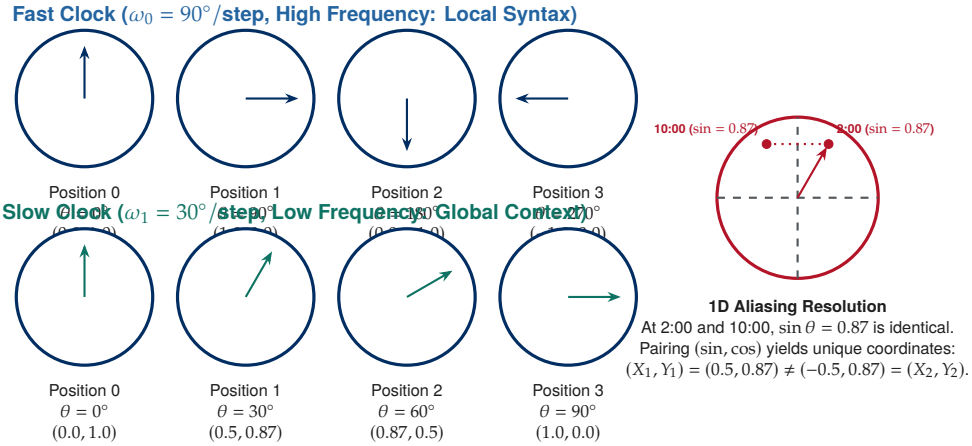


Figure 1.2: The geometric clock intuition for positional encodings. Each frequency channel pair acts as a rotating phasor. Fast clocks disambiguate immediate neighbors, while slow clocks track long-range sequences. Pairing sin and cos prevents 1D wave aliasing.

Numerical Calculations from Slide 35.
To illustrate the exact arithmetic, consider sequence position $pos = 1$ (the second token, under 0-indexed coordinates) with $d_{\text{model}} = 768$:

1. **Lowest channel ($i = 0$, even index $2i = 0$)**:

$$\text{PE}_{(1,0)} = \sin\left(\frac{1}{10000^{0/768}}\right) = \sin(1) \approx 0.8415. \tag{1.17}$$

2. **Lowest odd channel ($i = 0$, odd index $2i + 1 = 1$)**:

$$\text{PE}_{(1,1)} = \cos\left(\frac{1}{10000^{0/768}}\right) = \cos(1) \approx 0.5403. \tag{1.18}$$

3. **Intermediate channel ($i = 2$, even index $2i = 4$)**:

$$\text{PE}_{(1,4)} = \sin\left(\frac{1}{10000^{4/768}}\right) = \sin\left(\frac{1}{10000^{0.005208}}\right) \approx \sin\left(\frac{1}{1.0490}\right) = \sin(0.9533) \approx 0.8153. \tag{1.19}$$

These exact values $(0.8415, 0.5403, 0.8153)$ reproduce the values in Slide 35.
Theoretical Properties: Periodicity, Linear Shift, and Similarity.
Sinusoidal encodings exhibit three pivotal theoretical properties:

1. **Positional Uniqueness**: The constant base 10000 guarantees that within the first $2\pi \cdot 10000 \approx 62{,}832$ positions, the slowest frequency does not complete a single cycle, ensuring that every position vector in operational memory is globally unique.

2. **The Linear Relative Shift Property**: For any fixed spatial offset $k \in \mathbb{Z}$, the positional vector at position $pos + k$ can be computed as a direct linear transformation of the vector at position pos:

   Theorem 1.2: *Linear relative transformation property*

   For any frequency $\omega_i = 10000^{-2i/d_{\text{model}}}$ and spatial offset $k \in \mathbb{Z}$, there exists an orthogonal rotation matrix $\mathbf{M}_{i,k} \in \mathbb{R}^{2\times 2}$ independent of pos such that:

   $$\begin{bmatrix} \mathsf{PE}_{(pos+k,2i)} \\ \mathsf{PE}_{(pos+k,2i+1)} \end{bmatrix} = \begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix} \begin{bmatrix} \mathsf{PE}_{(pos,2i)} \\ \mathsf{PE}_{(pos,2i+1)} \end{bmatrix}. \tag{1.20}$$

   Proof. Applying the standard trigonometric angle addition identities:

   $$\sin(\omega_i(pos + k)) = \sin(\omega_i pos)\cos(\omega_i k) + \cos(\omega_i pos)\sin(\omega_i k), \tag{1.21}$$
   $$\cos(\omega_i(pos + k)) = \cos(\omega_i pos)\cos(\omega_i k) - \sin(\omega_i pos)\sin(\omega_i k). \tag{1.22}$$

   Expressing this system in matrix notation yields Equation (1.20). Because rotation matrix $\mathbf{M}_{i,k}$ depends strictly on offset k and is independent of pos, linear attention layers $(\mathbf{W}_Q, \mathbf{W}_K)$ can easily learn to attend to relative shifts. ■ □

3. **Cosine Similarity Preservation Across Distance**: Computing the inner product between positional vectors at positions p_1 and p_2 evaluates to:

   $$\mathbf{p}_{p_1}^{(i)} \cdot \mathbf{p}_{p_2}^{(i)} = \sin(\omega_i p_1)\sin(\omega_i p_2) + \cos(\omega_i p_1)\cos(\omega_i p_2) = \cos(\omega_i(p_1 - p_2)). \tag{1.23}$$

   The similarity depends strictly on relative displacement $|p_1 - p_2|$. As verified in Slides 38 and 39, the cosine similarity Gram matrix exhibits a bright main diagonal (cos = 1) with smooth monotonic decay as distance increases.

Advantages and Disadvantages of Sinusoidal Encodings.
Pros: Zero trainable parameters; theoretically capable of generalizing to arbitrary sequence lengths; provable linear shift properties.
Cons: Rigid, hand-crafted inductive bias; cannot adapt empirically to complex non-linear document structures or software syntax trees.

Learned absolute positional embeddings

An alternative paradigm, deployed in GPT-2, GPT-3, and BERT [**?** **?** **?**], treats positional encodings as trainable embedding parameters stored in a matrix:

$$\mathbf{E}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d_{\text{model}}}. \tag{1.24}$$

Each integer coordinate $pos \in \{0, 1, \ldots, L_{\max} - 1\}$ indexes row $\mathbf{p}_{pos} = (\mathbf{E}_{\text{pos}})_{pos,:} \in \mathbb{R}^{d_{\text{model}}}$, updated via gradient descent alongside token embeddings.

Advantages and Disadvantages of Learned Embeddings.
Pros: Highly flexible; automatically learns task-specific positional priors directly from data; empirical evaluations in [**?**] showed identical BLEU scores to sinusoidal encodings.
Cons: Hard context cutoff at $L_{\max}$. If an input sequence exceeds $L_{\max}$ tokens during inference, the model has no embedding vector for position $L_{\max}$, failing completely without architectural modifications or complex position interpolation techniques.

1.2.4 Relative positional embeddings (Shaw et al.)

In natural language and software engineering source code, the precise absolute coordinate of a token is frequently less important than its **relative displacement** with respect to other tokens. Whether an active verb appears at index 12 or index 84, its grammatical subject is typically found 1 or 2 tokens earlier.

To incorporate this inductive bias directly into the attention mechanism, Shaw et al. [**?**] formulated **Relative Positional Representations**. Instead of adding absolute vectors to input token embeddings, relative positional embeddings inject pairwise distance information directly into the scaled dot-product attention logits (Slides 78–79):

$$\texttt{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top + \mathbf{S}_{\text{rel}}}{\sqrt{d_k}}\right)\mathbf{V}, \tag{1.25}$$

where $\mathbf{S}_{\text{rel}} \in \mathbb{R}^{L\times L}$ is a relative position bias matrix. For query position i and key position j, the entry is defined by:

$$S_{\text{rel}}[i, j] = \mathbf{q}_i^\top \mathbf{a}^K_{j-i}, \tag{1.26}$$

where $\mathbf{a}^K_{j-i} \in \mathbb{R}^{d_k}$ is a learned vector representing the relative distance $\delta = j - i$.

To bound parameter count, relative positions are clipped to a maximum horizon $k_{\max}$:

$$\text{clip}(\delta, k_{\max}) = \max(-k_{\max}, \min(k_{\max}, \delta)). \tag{1.27}$$

Table 1.4 reproduces the relative coordinate matrix from Slide 78 across a 6-token sequence.

Table 1.4: Relative displacement matrix $\delta = j - i$ between queries (rows i) and keys (columns j).

Query \ Key	Key 1	Key 2	Key 3	Key 4	Key 5	Key 6
Query 1	0	+1	+2	+3	+4	+5
Query 2	−1	0	+1	+2	+3	+4
Query 3	−2	−1	0	+1	+2	+3
Query 4	−3	−2	−1	0	+1	+2
Query 5	−4	−3	−2	−1	0	+1
Query 6	−5	−4	−3	−2	−1	0

Asymmetry: why relative embeddings fit self-attention but not cross-attention

As emphasized in Slide 79, relative positional embeddings are mathematically natural for **self-attention**, but fundamentally problematic for **cross-attention**:

- In self-attention, queries and keys reside in the same physical sequence. Therefore, spatial difference $\delta = j - i$ has an unambiguous physical and topological meaning (e.g., "2 tokens before the query").
- In encoder-decoder cross-attention, queries originate from the target language sequence (the decoder) while keys originate from the source language sequence (the encoder).

Because linguistic word orders diverge substantially between languages (for example, English Subject-Verb-Object versus Japanese Subject-Object-Verb), there is no monotonic 1-to-1 linear correspondence between source and target indices. Computing a scalar distance $j_{\text{source}} - i_{\text{target}}$ has no meaningful linguistic or topological interpretation.

Consequently, cross-attention must rely on absolute representations or content-based addressing, whereas self-attention can deploy relative representations effectively.

1.3 The Attention mechanism: from dictionary lookups to scaled dot products

1.3.1 Motivation: dynamic context routing vs. static aggregation

As established in Chapter **??**, static word embeddings map discrete words to fixed vector coordinates $\mathbf{w} \in \mathbb{R}^d$. However, natural language is inherently polysemous and context-dependent: the noun *"bank"* occupies radically different semantic realms in *"the bank approved the mortgage"* versus *"the grassy bank of the river"*.

Prior to the Transformer, recurrent neural networks attempted to resolve context by recursively updating a hidden state $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$. Yet in sequence-to-sequence encoder-decoder networks, compressing an entire input sequence into a single static context vector $\mathbf{h}_L$ induces an information compression bottleneck that destroys fine-grained details on long sequences.

The **Attention Mechanism** overcomes both limitations simultaneously. Rather than compressing context into a single static vector, attention allows every token representation to be dynamically synthesized at run-time as an input-dependent weighted combination of all other tokens in the sequence. Each token's representation is contextualized by dynamically routing information from the most relevant words in its surrounding environment.

1.3.2 The associative memory progression: dictionary lookup to continuous attention

To understand the mathematical mechanics of attention from first principles, it is instructive to trace its conceptual evolution from classical computer science data structures (Slides 48–52).

1. Classical dictionary lookup (discrete hash table)

In standard computer systems, a dictionary, associative array, or hash map stores information as a collection of M discrete key-value pairs:

$$\mathcal{D} = \{(k_1, v_1), (k_2, v_2), \dots, (k_M, v_M)\}. \tag{1.28}$$

When a discrete search query q is submitted, the hash map evaluates an exact binary match against all keys:

$$\text{Lookup}(q, \mathcal{D}) = \begin{cases} v_m & \text{if } \exists\, m \text{ such that } q = k_m, \\ \texttt{None} & \text{otherwise.} \end{cases} \tag{1.29}$$

This classical retrieval is binary (0 or 1) and non-differentiable: if query q differs from key k_m by a single character or bit, the match fails completely.

2. Continuous dot-product matching (soft alignment)

Attention generalizes hash-table retrieval to continuous vector spaces:

- A **Query** vector $\mathbf{q} \in \mathbb{R}^{d_k}$ represents the search criterion: *"What information am I seeking?"*
- A set of M **Key** vectors $\mathbf{k}_j \in \mathbb{R}^{d_k}$ represent content labels: *"What features do I describe?"*
- A set of M **Value** vectors $\mathbf{v}_j \in \mathbb{R}^{d_v}$ represent the informative payload: *"What content do I supply?"*

Instead of checking binary equality $q = k_j$, we evaluate a continuous compatibility score $s_j \in \mathbb{R}$ using vector inner products (Slide 50):

$$s_j = \mathbf{q}^\top \mathbf{k}_j. \tag{1.30}$$

The dot product quantifies the directional alignment and projection of the query onto the key in the latent feature space.

3. Softmax probability normalization

Because raw inner products $s_j \in (-\infty, +\infty)$ are unbounded, we map them onto the probability simplex using the softmax function (Slide 52):

$$\alpha_j = \frac{\exp(s_j)}{\sum_{l=1}^{M} \exp(s_l)}, \quad \text{satisfying} \quad \alpha_j \in [0,1] \quad \text{and} \quad \sum_{j=1}^{M} \alpha_j = 1. \tag{1.31}$$

The scalar α_j represents the fractional attention weight allocated by query $\mathbf{q}$ to key $\mathbf{k}_j$.

4. Contextual value aggregation

Finally, the retrieved contextual vector $\mathbf{z}$ is computed as the weighted convex combination of all value vectors (Slide 50):

$$\mathbf{z} = \sum_{j=1}^{M} \alpha_j \mathbf{v}_j \in \mathbb{R}^{d_v}. \tag{1.32}$$

Because vector inner products, exponentials, and summations are smooth, differentiable operators, the entire retrieval mechanism can be trained end-to-end via gradient backpropagation.

1.3.3 Visual and geometric intuition: 3Blue1Brown projection perspective

To build a deep geometric intuition for self-attention, we examine the visual perspective popularized by Grant Sanderson (3Blue1Brown) [**?**].

Consider tokens residing in a high-dimensional continuous embedding space $\mathbb{R}^{d_{\text{model}}}$. Initially, each token begins as a static context-free embedding vector $\mathbf{x}_i$. To allow tokens to communicate, the model introduces three learned linear transformation matrices:

$$\mathbf{W}_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}. \tag{1.33}$$

Geometrically, these matrices act as specialized feature lenses:

1. **Queries ($\mathbf{q}_i = \mathbf{x}_i \mathbf{W}_Q$)**: Represent a directional question or request formulated by token i. For example, an active verb projects a query asking: *"Where is my direct object noun?"*

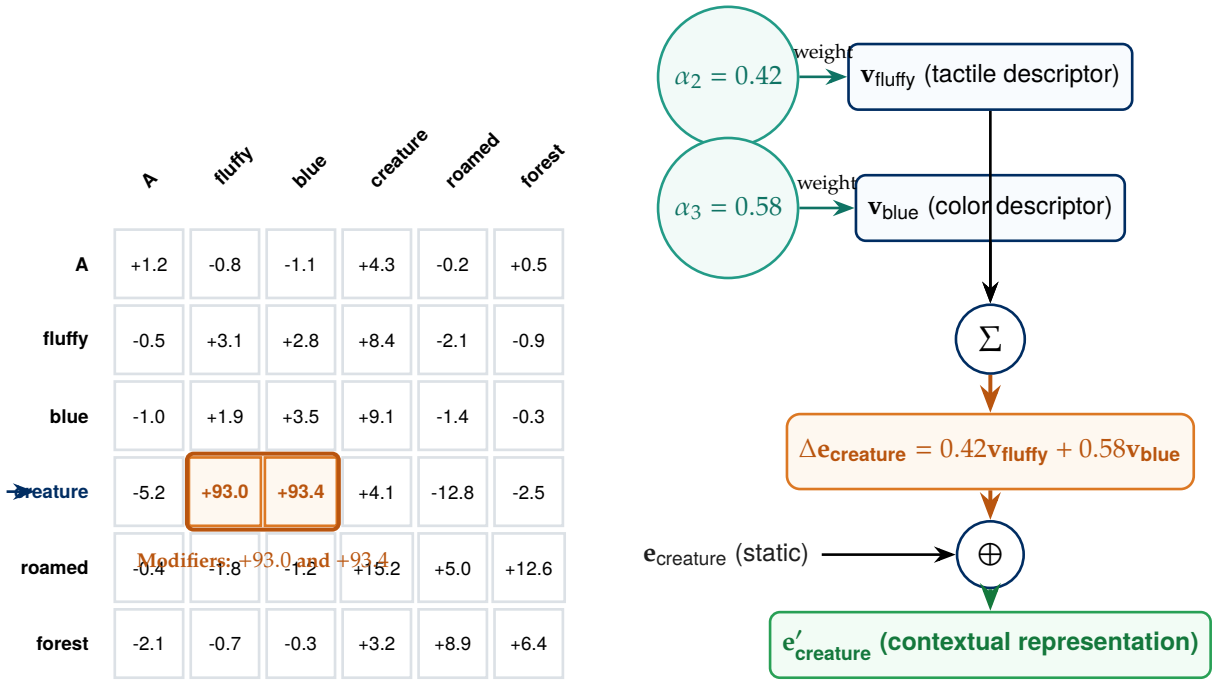
2. **Keys ($\mathbf{k}_j = \mathbf{x}_j \mathbf{W}_K$)**: Represent feature advertisements broadcast by token j. A noun projects a key advertising: *"I am a direct object noun preceding you."*

3. **Inner Product ($\mathbf{q}_i^\top \mathbf{k}_j$)**: Geometrically computes the dot product between the query vector and the key vector. When a query and a key point in aligned directions in feature space ($\theta \approx 0°$), their inner product is large and positive, indicating high semantic or syntactic affinity.

4. **Values ($\mathbf{v}_j = \mathbf{x}_j \mathbf{W}_V$)**: Represent the substantive informational payload that token j transmits to any token that attends to it.

5. **Contextual Update ($\Delta \mathbf{x}_i$)**: The weighted sum of values $\sum_j \alpha_{ij} \mathbf{v}_j$ produces a directional displacement vector. When added back into the residual stream, it physically displaces token i's coordinate in embedding space toward the specialized manifold region representing its contextualized meaning.

Figure 1.3 illustrates this geometric dynamic on the phrase *"A fluffy blue creature roamed the verdant forest"*.



(a) Query-Key compatibility matrix ($\mathbf{QK}^\top$) (b) Value matrix weighting and vector update ($\Delta\mathbf{e}$)

Figure 1.3: Geometric attention dynamics and matrix transformations in 3Blue1Brown's pedagogical breakdown [**?**]. (a) Pairwise Query-Key dot products evaluate compatibility across all token pairs; the Query for *"creature"* strongly matches the Keys for descriptive modifiers *"fluffy"* (+93.0) and *"blue"* (+93.4). (b) The attention table routes the corresponding Value vectors: after softmax normalization (0.42 and 0.58), the weighted sum produces a displacement vector $\Delta\mathbf{E}_4 = 0.42\mathbf{V}_2 + 0.58\mathbf{V}_3$, shifting the representation into the specialized manifold coordinates of a fluffy blue creature.

1.3.4 Scaled Dot-Product Attention and theoretical variance derivation

Packing all N queries, M keys, and M values into matrices $\mathbf{Q} \in \mathbb{R}^{N \times d_k}$, $\mathbf{K} \in \mathbb{R}^{M \times d_k}$, and $\mathbf{V} \in \mathbb{R}^{M \times d_v}$ yields the canonical matrix formulation of **Scaled Dot-Product Attention** [**?**] (Slide 55):

$$\texttt{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}, \tag{1.34}$$

where $\mathbf{M} \in \{0, -\infty\}^{N \times M}$ is an optional masking matrix, and softmax is applied row-wise.

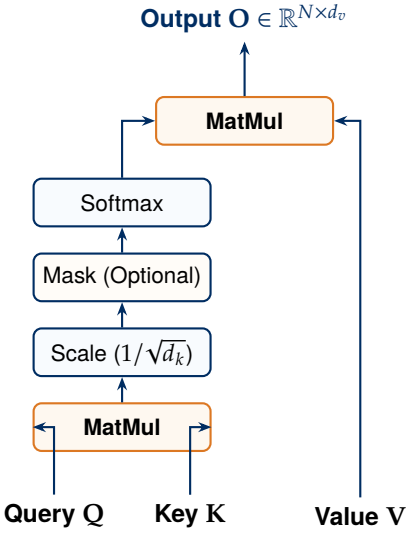


Figure 1.4: Computational dataflow of Scaled Dot-Product Attention [**?**].

Theoretical derivation: why the $\frac{1}{\sqrt{d_k}}$ scaling factor is mandatory

A central mathematical innovation of Vaswani et al. is the scaling factor $\frac{1}{\sqrt{d_k}}$. To prove why this scaling is mathematically indispensable, we analyze the statistical behavior of inner products in high dimensions.

Theorem 1.3: *Variance of high-dimensional dot products and softmax gradient vanishing*

Assume the components of query vector $\mathbf{q} = [q_1, \ldots, q_{d_k}]^\top$ and key vector $\mathbf{k} = [k_1, \ldots, k_{d_k}]^\top$ are independent and identically distributed random variables with mean zero and unit variance:

$$\mathbb{E}[q_m] = \mathbb{E}[k_m] = 0, \quad \mathbb{V}\text{ar}(q_m) = \mathbb{V}\text{ar}(k_m) = 1, \quad \forall m \in \{1, 2, \ldots, d_k\}. \tag{1.35}$$

Then the dot product $s = \mathbf{q}^\top \mathbf{k} = \sum_{m=1}^{d_k} q_m k_m$ has mean zero and variance d_k:

$$\mathbb{E}[s] = 0, \quad \mathbb{V}\text{ar}(s) = d_k. \tag{1.36}$$

Furthermore, as $d_k \to \infty$, the unscaled scores saturate the softmax operator, causing backpropagation gradients to vanish exponentially:

$$\lim_{|s| \to \infty} \frac{\partial \, \text{softmax}_i(\mathbf{s})}{\partial s_j} = 0. \tag{1.37}$$

Scaling by $\frac{1}{\sqrt{d_k}}$ preserves unit variance $\mathbb{V}\text{ar}\left(\frac{s}{\sqrt{d_k}}\right) = 1$ independently of dimension d_k, ensuring healthy, non-vanishing gradient flow.

Proof. By the linearity of expectation and independence:

$$\mathbb{E}[s] = \mathbb{E}\left[\sum_{m=1}^{d_k} q_m k_m\right] = \sum_{m=1}^{d_k} \mathbb{E}[q_m]\,\mathbb{E}[k_m] = \sum_{m=1}^{d_k} 0 \cdot 0 = 0. \tag{1.38}$$

For the variance, since the individual product terms $z_m = q_m k_m$ are mutually independent:

$$\mathbb{V}\mathrm{ar}(s) = \mathbb{V}\mathrm{ar}\left(\sum_{m=1}^{d_k} q_m k_m\right) = \sum_{m=1}^{d_k} \mathbb{V}\mathrm{ar}(q_m k_m). \tag{1.39}$$

Using the identity $\mathbb{V}\mathrm{ar}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$:

$$\mathbb{V}\mathrm{ar}(q_m k_m) = \mathbb{E}[q_m^2 k_m^2] - (\mathbb{E}[q_m k_m])^2 = \mathbb{E}[q_m^2]\,\mathbb{E}[k_m^2] - 0 = (1)(1) = 1. \tag{1.40}$$

Summing across all d_k dimensions:

$$\mathbb{V}\mathrm{ar}(s) = \sum_{m=1}^{d_k} 1 = d_k. \tag{1.41}$$

The standard deviation of the raw dot product is therefore $\sigma_s = \sqrt{d_k}$. For a typical projection dimension $d_k = 64$, standard deviation is $\sigma_s = 8$. Scores frequently take values in $[-24, +24]$.

Now consider the derivative of the softmax function $\sigma_i = \mathrm{softmax}_i(\mathbf{s})$:

$$\frac{\partial \sigma_i}{\partial s_j} = \sigma_i(\delta_{ij} - \sigma_j). \tag{1.42}$$

When the inputs $\mathbf{s}$ have large magnitudes, the largest element $s_{\max}$ dominates the exponent sum:

$$\sigma_{\max} = \frac{\exp(s_{\max})}{\sum_l \exp(s_l)} \approx 1, \quad \sigma_j \approx 0 \quad (\forall j \neq \max). \tag{1.43}$$

Evaluating the Jacobian in Equation (1.42) under this saturation regime:

$$\text{For } i = j = \max: \quad \frac{\partial \sigma_{\max}}{\partial s_{\max}} = \sigma_{\max}(1 - \sigma_{\max}) \approx 1(1-1) = 0, \tag{1.44}$$

$$\text{For } i \neq j: \quad \frac{\partial \sigma_i}{\partial s_j} = -\sigma_i \sigma_j \approx -(0)(0) = 0. \tag{1.45}$$

All partial derivatives vanish completely. The model enters a plateau where gradient backpropagation cannot adjust the projection weights $\mathbf{W}_Q$ and $\mathbf{W}_K$.

Dividing by the scalar $\sqrt{d_k}$ normalizes the variance:

$$\mathbb{V}\mathrm{ar}\left(\frac{s}{\sqrt{d_k}}\right) = \frac{1}{(\sqrt{d_k})^2}\,\mathbb{V}\mathrm{ar}(s) = \frac{1}{d_k}(d_k) = 1. \tag{1.46}$$

Regardless of how large d_k is chosen, the variance of the scaled attention scores $\tilde{s} = \frac{s}{\sqrt{d_k}}$ remains strictly unitary, maintaining the logits within the linear, steep gradient regime of the softmax function. ■ □

1.3.5 Complete numerical worked example: reproducing the lecture calculations

To illustrate the matrix operations with complete numerical transparency, we reproduce the step-by-step worked example from Slides 57 through 65, tracking every calculation down to four decimal places.

Consider an input sequence of $N = 3$ tokens in a 2-dimensional latent embedding space ($d_{\text{model}} = 2$), with projection dimensions $d_k = d_v = 2$:

$$\mathbf{X} = \begin{bmatrix} 0.1 & 0.5 \\ 0.1 & 0.2 \\ 0.9 & 0.9 \end{bmatrix} \in \mathbb{R}^{3\times 2}, \quad \mathbf{W}_Q = \begin{bmatrix} 0.9 & 0.9 \\ 0.5 & 0.3 \end{bmatrix}, \quad \mathbf{W}_K = \begin{bmatrix} 1.0 & 0.9 \\ 1.0 & 0.6 \end{bmatrix}, \quad \mathbf{W}_V = \begin{bmatrix} 0.8 & 0.9 \\ 0.9 & 1.0 \end{bmatrix}. \tag{1.47}$$

Step 1: Linear projection into Queries, Keys, and Values.
Multiplying $\mathbf{X}$ by each projection matrix yields:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q = \begin{bmatrix} 0.1(0.9) + 0.5(0.5) & 0.1(0.9) + 0.5(0.3) \\ 0.1(0.9) + 0.2(0.5) & 0.1(0.9) + 0.2(0.3) \\ 0.9(0.9) + 0.9(0.5) & 0.9(0.9) + 0.9(0.3) \end{bmatrix} = \begin{bmatrix} 0.34 & 0.24 \\ 0.19 & 0.15 \\ 1.26 & 1.08 \end{bmatrix}, \tag{1.48}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_K = \begin{bmatrix} 0.1(1.0) + 0.5(1.0) & 0.1(0.9) + 0.5(0.6) \\ 0.1(1.0) + 0.2(1.0) & 0.1(0.9) + 0.2(0.6) \\ 0.9(1.0) + 0.9(1.0) & 0.9(0.9) + 0.9(0.6) \end{bmatrix} = \begin{bmatrix} 0.60 & 0.39 \\ 0.30 & 0.21 \\ 1.80 & 1.35 \end{bmatrix}, \tag{1.49}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_V = \begin{bmatrix} 0.1(0.8) + 0.5(0.9) & 0.1(0.9) + 0.5(1.0) \\ 0.1(0.8) + 0.2(0.9) & 0.1(0.9) + 0.2(1.0) \\ 0.9(0.8) + 0.9(0.9) & 0.9(0.9) + 0.9(1.0) \end{bmatrix} = \begin{bmatrix} 0.53 & 0.59 \\ 0.26 & 0.29 \\ 1.53 & 1.71 \end{bmatrix}. \tag{1.50}$$

Step 2: Unscaled dot-product computation.
Evaluating the matrix multiplication $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{3\times 3}$:

$$\mathbf{Q}\mathbf{K}^\top = \begin{bmatrix} 0.34 & 0.24 \\ 0.19 & 0.15 \\ 1.26 & 1.08 \end{bmatrix} \begin{bmatrix} 0.60 & 0.30 & 1.80 \\ 0.39 & 0.21 & 1.35 \end{bmatrix} = \begin{bmatrix} 0.2976 & 0.1524 & 0.9360 \\ 0.1725 & 0.0885 & 0.5445 \\ 1.1772 & 0.6048 & 3.7260 \end{bmatrix}. \tag{1.51}$$

Focusing on the first token ($\mathbf{q}_1 = [0.34, 0.24]$), the individual dot products evaluate to (Slides 61–63):

$$s_{1,1} = \mathbf{q}_1^\top \mathbf{k}_1 = 0.34(0.60) + 0.24(0.39) = 0.2040 + 0.0936 = 0.2976, \tag{1.52}$$

$$s_{1,2} = \mathbf{q}_1^\top \mathbf{k}_2 = 0.34(0.30) + 0.24(0.21) = 0.1020 + 0.0504 = 0.1524, \tag{1.53}$$

$$s_{1,3} = \mathbf{q}_1^\top \mathbf{k}_3 = 0.34(1.80) + 0.24(1.35) = 0.6120 + 0.3240 = 0.9360. \tag{1.54}$$

Step 3: Scaling by $\frac{1}{\sqrt{d_k}} = \frac{1}{\sqrt{2}} \approx 0.7071$.
Dividing row 1 by $\sqrt{2}$ yields the scaled scores (Slide 64):

$$\tilde{s}_{1,1} = \frac{0.2976}{\sqrt{2}} \approx 0.2104, \tag{1.55}$$

$$\tilde{s}_{1,2} = \frac{0.1524}{\sqrt{2}} \approx 0.1078, \tag{1.56}$$

$$\tilde{s}_{1,3} = \frac{0.9360}{\sqrt{2}} \approx 0.6619. \tag{1.57}$$

Step 4: Softmax probability normalization.
Exponentiating and normalizing row 1 across the three keys:

$$\exp(0.2104) \approx 1.2342, \quad \exp(0.1078) \approx 1.1138, \quad \exp(0.6619) \approx 1.9385, \tag{1.58}$$

$$\text{Partition Sum } Z_1 = 1.2342 + 1.1138 + 1.9385 = 4.2865, \tag{1.59}$$

$$\alpha_{1,1} = \frac{1.2342}{4.2865} \approx 0.2879, \tag{1.60}$$

$$\alpha_{1,2} = \frac{1.1138}{4.2865} \approx 0.2598, \tag{1.61}$$

$$\alpha_{1,3} = \frac{1.9385}{4.2865} \approx 0.4522. \tag{1.62}$$

Notice that $\sum_{j=1}^{3} \alpha_{1,j} = 0.2879 + 0.2598 + 0.4522 = 0.9999 \approx 1.0000$.
Step 5: Weighted value aggregation.
The contextual output vector $\mathbf{o}_1$ for token 1 is computed as the linear combination of values (Slide 65):

$$\begin{aligned} \mathbf{o}_1 &= \alpha_{1,1}\mathbf{v}_1 + \alpha_{1,2}\mathbf{v}_2 + \alpha_{1,3}\mathbf{v}_3 \\ &= 0.2879[0.53, 0.59]^\top + 0.2598[0.26, 0.29]^\top + 0.4522[1.53, 1.71]^\top \\ &= [0.1526, 0.1699]^\top + [0.0675, 0.0753]^\top + [0.6919, 0.7733]^\top \\ &\approx [0.9120, 1.0185]^\top \approx [0.91, 1.02]^\top. \end{aligned} \tag{1.63}$$

Evaluating all three rows simultaneously via matrix multiplication $\mathbf{O} = \mathbf{AV} \in \mathbb{R}^{3\times 2}$ yields:

$$\mathbf{O} = \begin{bmatrix} 0.2879 & 0.2598 & 0.4522 \\ 0.3083 & 0.2906 & 0.4011 \\ 0.1294 & 0.0863 & 0.7843 \end{bmatrix} \begin{bmatrix} 0.53 & 0.59 \\ 0.26 & 0.29 \\ 1.53 & 1.71 \end{bmatrix} \approx \begin{bmatrix} 0.91 & 1.02 \\ 0.85 & 0.95 \\ 1.29 & 1.44 \end{bmatrix}. \tag{1.64}$$

This reproduces the exact tensor calculations displayed in the lecture slides.

1.4 Attention types across architectures: self-attention, masked attention, and cross-attention

1.4.1 Architectural motivation for diverse attention types

Within the full Transformer architecture, attention is not applied in a monolithic, uniform configuration. Depending on whether a sublayer is performing **representation extraction**, **causal sequence generation**, or **cross-lingual conditioning**, the visibility of tokens and the sources of queries, keys, and values must satisfy distinct structural constraints:

1. **Representation Learning (Encoder)**: Requires an unconstrained bidirectional receptive field where every token can freely inspect its complete surrounding linguistic context (both left and right).

2. **Autoregressive Generation (Decoder)**: Mandates strict causal isolation, guaranteeing that the model cannot peek at future tokens during generation or training.

3. **Sequence-to-Sequence Grounding (Encoder-Decoder)**: Requires a cross-attention bridge that allows target-language decoder tokens to query and extract information dynamically from the source-language encoder memory.

Figure 1.5 illustrates the structural distinction across these three attention mechanisms.

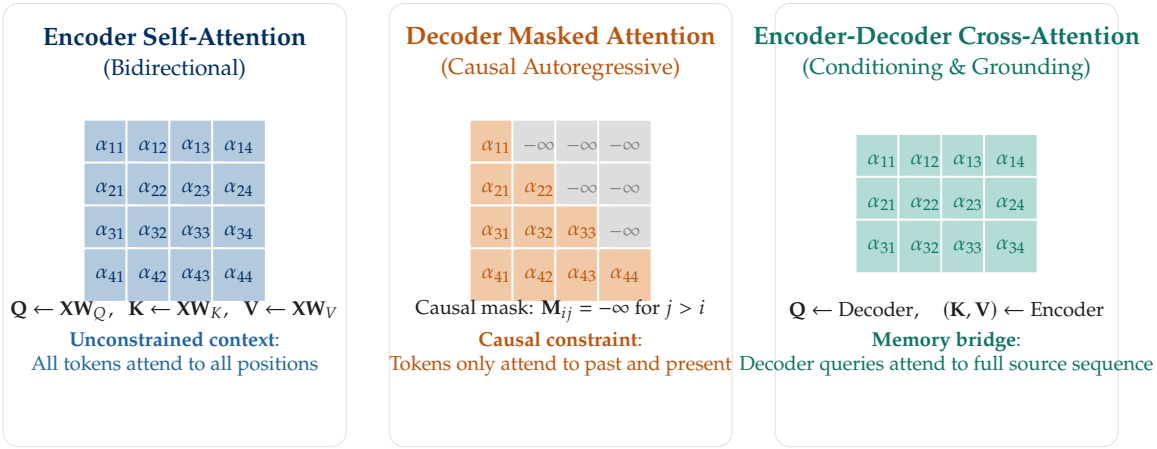


Figure 1.5: The triad of attention mechanisms across the Transformer architecture: (a) Bidirectional Encoder Self-Attention, (b) Causal Decoder Masked Self-Attention, and (c) Encoder-Decoder Cross-Attention.

1.4.2 Bidirectional Encoder Self-Attention

General idea

In the Transformer Encoder, the primary objective is producing uncompressed, deeply contextualized representations of the input sequence. Because the encoder does not perform autoregressive generation, there is no risk of future information leakage. Therefore, every token is granted an unconstrained, bidirectional receptive field: token i can attend to tokens in the past ($j < i$), the present ($j = i$), and the future ($j > i$).

Mathematical formulation

Queries, keys, and values all derive from the exact same input tensor $\mathbf{X} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$ via linear projections:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \tag{1.65}$$

where $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. No masking matrix is applied ($\mathbf{M} = \mathbf{0}$). The bidirectional attention operator evaluates:

$$\mathbf{A}_{\text{enc}} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right), \quad \mathbf{O}_{\text{enc}} = \mathbf{A}_{\text{enc}}\mathbf{V} \in \mathbb{R}^{L_{\text{enc}} \times d_v}. \tag{1.66}$$

Every element $\alpha_{ij} > 0$, enabling full all-to-all information exchange.

Concrete natural language example: polysemy disambiguation

Consider the sentence: *"The bank of the river was overflowing."* The word *"bank"* is polysemous (financial institution vs. river embankment). When computing the representation for *"bank"* (index 2), an unconstrained bidirectional encoder computes queries that match against the key of the future token *"river"* (index 5). The resulting high attention weight $\alpha_{2,5}$ routes the value features of *"river"* into the output representation of *"bank"*, resolving its meaning into a geological riverbank.

1.4.3 Decoder Masked Causal Self-Attention

General idea

In autoregressive text generation, the probability distribution of the next token y_t depends strictly on previously generated tokens: $P(y_t \mid y_1, \ldots, y_{t-1})$. During training, modern hardware processes all target tokens simultaneously in a single forward pass using **Teacher Forcing**. However, if token t were allowed to attend to tokens $t+1, \ldots, T$, it would trivially copy the target token, destroying the generative learning task.

To prevent future information leakage while preserving full parallel training efficiency, the decoder applies **Causal Masking**.

Mathematical formulation

Causal masking adds an upper-triangular mask matrix $\mathbf{M} \in \mathbb{R}^{L_{\text{dec}} \times L_{\text{dec}}}$ to the unscaled query-key logits prior to the softmax operator (Slide 70):

$$M_{ij} = \begin{cases} 0 & \text{if } j \le i \quad \text{(past and present tokens: valid)}, \\ -\infty & \text{if } j > i \quad \text{(future tokens: masked)}. \end{cases} \tag{1.67}$$

The masked self-attention operator evaluates:

$$\mathbf{A}_{\text{dec}} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right), \quad \mathbf{O}_{\text{dec}} = \mathbf{A}_{\text{dec}}\mathbf{V} \in \mathbb{R}^{L_{\text{dec}} \times d_v}. \tag{1.68}$$

Because $\exp(-\infty) = 0$, applying the softmax operator along row i yields:

$$\alpha_{ij} = \frac{\exp(s_{ij} + M_{ij})}{\sum_{l=1}^{L_{\text{dec}}} \exp(s_{il} + M_{il})} = \begin{cases} \frac{\exp(s_{ij})}{\sum_{l=1}^{i} \exp(s_{il})} & \text{for } j \le i, \\ 0 & \text{for } j > i. \end{cases} \tag{1.69}$$

Future positions receive strictly zero attention weight. As noted in Slide 70, computing all pairwise dot products via matrix multiplication and masking invalid entries is dramatically faster on GPUs than executing sequential loops.

Concrete worked example: autoregressive generation under causal masking

Consider an autoregressive target sequence of $T = 3$ tokens: `["<BOS>", "The", "cat"]`. Suppose scaled query-key logits evaluate to:

$$\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} = \begin{bmatrix} 1.2 & 0.8 & 0.4 \\ 0.5 & 1.5 & 1.1 \\ 0.3 & 0.9 & 2.1 \end{bmatrix}. \tag{1.70}$$

Adding the causal mask $\mathbf{M}$ produces:

$$\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M} = \begin{bmatrix} 1.2 & -\infty & -\infty \\ 0.5 & 1.5 & -\infty \\ 0.3 & 0.9 & 2.1 \end{bmatrix}. \tag{1.71}$$

Evaluating the row-wise softmax yields the strictly lower-triangular attention matrix:

$$\mathbf{A}_{\text{masked}} = \begin{bmatrix} 1.0000 & 0.0000 & 0.0000 \\ 0.2689 & 0.7311 & 0.0000 \\ 0.1171 & 0.2133 & 0.6696 \end{bmatrix}. \tag{1.72}$$

Token 1 (`"<BOS>"`) attends exclusively to itself with weight 1.0. Token 2 (`"The"`) attends to `"<BOS>"` (0.27) and itself (0.73), but is strictly isolated from `"cat"` (0.0). Causal autoregressive integrity is rigorously preserved.

1.4.4 Encoder-Decoder Cross-Attention

General idea

Encoder-Decoder Cross-Attention acts as the cognitive bridge linking the target-generation decoder to the source-encoding encoder (Slide 71). It enables the decoder to dynamically query the full source sequence representation at each step of text generation.

Mathematical formulation and asymmetric dimensions

In cross-attention, the inputs are strictly decoupled:

- **Queries (Q)**: Linearly projected from the decoder's prior sublayer representations $\mathbf{H}_{\text{dec}} \in \mathbb{R}^{L_{\text{dec}} \times d_{\text{model}}}$:
$$\mathbf{Q} = \mathbf{H}_{\text{dec}}\mathbf{W}_Q \in \mathbb{R}^{L_{\text{dec}} \times d_k}. \tag{1.73}$$
- **Keys (K) and Values (V)**: Linearly projected from the terminal representations of the encoder stack $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$:
$$\mathbf{K} = \mathbf{H}_{\text{enc}}\mathbf{W}_K \in \mathbb{R}^{L_{\text{enc}} \times d_k}, \quad \mathbf{V} = \mathbf{H}_{\text{enc}}\mathbf{W}_V \in \mathbb{R}^{L_{\text{enc}} \times d_v}. \tag{1.74}$$

The cross-attention operator evaluates:

$$\mathbf{A}_{\text{cross}} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{L_{\text{dec}} \times L_{\text{enc}}}, \quad \mathbf{O}_{\text{cross}} = \mathbf{A}_{\text{cross}}\mathbf{V} \in \mathbb{R}^{L_{\text{dec}} \times d_v}. \tag{1.75}$$

Critical Dimensional and Masking Properties:

1. **Asymmetric sequence lengths**: The source sequence length L_{enc} and target sequence length L_{dec} do not need to be equal ($L_{\text{dec}} \neq L_{\text{enc}}$). The query-key product produces an $L_{\text{dec}} \times L_{\text{enc}}$ alignment matrix, which multiplies $\mathbf{V} \in \mathbb{R}^{L_{\text{enc}} \times d_v}$ to yield an output tensor matching the decoder's length $\mathbb{R}^{L_{\text{dec}} \times d_v}$.
2. **Zero causal masking**: Because the encoder has already completely processed the source sequence, the entire source text is available. The decoder can attend across all encoder positions without causal masking (Slide 71).

Concrete worked example: cross-lingual machine translation

Consider translating English into French:

$$\textbf{Source}: \texttt{["The", "black", "cat"]}\ (L_{\text{enc}} = 3) \implies \textbf{Target}: \texttt{["Le", "chat", "noir"]}\ (L_{\text{dec}} = 3).$$

When the decoder is positioned at token 2 (`"chat"`), its query vector $\mathbf{q}_{\text{chat}}$ searches for the corresponding source entity. The query computes inner products against the encoder keys:

$$\mathbf{q}_{\text{chat}}^{\top}\mathbf{k}_{\text{The}} = 0.5, \quad \mathbf{q}_{\text{chat}}^{\top}\mathbf{k}_{\text{black}} = 1.2, \quad \mathbf{q}_{\text{chat}}^{\top}\mathbf{k}_{\text{cat}} = 6.8. \tag{1.76}$$

Applying softmax allocates over 99% of the attention weight to the source key $\mathbf{k}_{\text{cat}}$. Cross-attention dynamically extracts the value payload $\mathbf{v}_{\text{cat}}$, allowing the decoder to generate `"chat"` conditioned directly on the source noun.

1.5 Transformer core blocks and end-to-end processing journey

1.5.1 Multi-Head Attention (MHA)

Motivation: why a single attention head is insufficient

A single scaled dot-product attention head computes a single weighted average over values for each token. However, in natural language syntax and semantics, a single word simultaneously participates in multiple, distinct linguistic relationships (Slide 72).

Consider the word *"making"* in the sentence *"The programmer was making rapid progress on the compiler"*. The token *"making"* simultaneously participates in:

1. **Syntactic subject agreement**: Connecting back to the subject *"programmer"*.
2. **Direct object selection**: Connecting forward to the nominal object *"progress"*.
3. **Adverbial modification**: Linking with the adjective modifier *"rapid"*.
4. **Prepositional attachment**: Grounding into the prepositional phrase *"on the compiler"*.

If forced through a single attention head, all these disparate relational vectors are blended together into a single partition sum, diluting specialized information.

Multi-Head Attention resolves this representational bottleneck by projecting queries, keys, and values into h distinct, lower-dimensional representation subspaces in parallel (Slides 72–73). Each attention head is parameterized by its own projection matrices, allowing individual heads to specialize in different linguistic patterns (e.g., Head 1 tracks syntax, Head 2 resolves coreference, Head 3 models semantic associations).

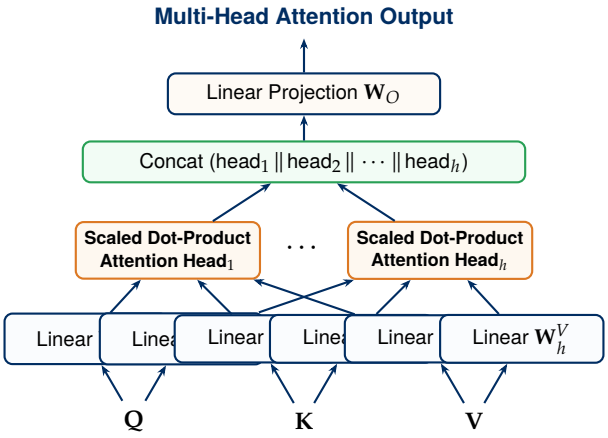


Figure 1.6: Architecture of Multi-Head Attention (MHA) [**?**]. Queries, Keys, and Values are projected into h parallel representation subspaces, evaluated independently via Scaled Dot-Product Attention, concatenated, and projected through $\mathbf{W}_O$.

Mathematical formulation

Let h denote the number of parallel attention heads. In the canonical architecture, projection dimensions are scaled such that $d_k = d_v = d_{\text{model}}/h$. For example, with $d_{\text{model}} = 512$ and $h = 8$, each head operates on subspace dimension $d_k = 64$.

For each head $i \in \{1, 2, \ldots, h\}$, projections are parameterized by learned matrices:

$$\mathbf{W}_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}. \tag{1.77}$$

Each individual head evaluates scaled dot-product attention in its designated subspace:

$$\text{head}_i = \texttt{Attention}\left(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V\right) \in \mathbb{R}^{L\times d_v}. \tag{1.78}$$

The outputs of all h heads are concatenated along the feature dimension and projected back to the model dimension d_{model} using an output projection matrix $\mathbf{W}_O \in \mathbb{R}^{hd_v\times d_{\text{model}}}$:

$$\texttt{MultiHead}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h)\,\mathbf{W}_O. \tag{1.79}$$

Parameter and computational invariance

Crucially, Multi-Head Attention incurs the **exact same computational and parameter cost** as a single full-dimensional attention head:

- In a single head operating on d_{model}: The projection parameter count is $3\times(d_{\text{model}}\times d_{\text{model}}) = 3d_{\text{model}}^2$.
- In Multi-Head Attention with h heads of dimension $d_k = d_{\text{model}}/h$: Each head requires $3\times(d_{\text{model}}\times\frac{d_{\text{model}}}{h}) = \frac{3d_{\text{model}}^2}{h}$ parameters. Summing across all h heads yields: $h\times\frac{3d_{\text{model}}^2}{h} = 3d_{\text{model}}^2$. Adding the output projection $\mathbf{W}_O \in \mathbb{R}^{d_{\text{model}}\times d_{\text{model}}}$ (d_{model}^2 parameters) produces identical parameter footprints.

Due to reduced subspace dimensions ($d_k = d_{\text{model}}/h$), total compute time is comparable to single-head attention while achieving vastly superior representational diversity.

1.5.2 Position-wise Feed-Forward Networks (FFN)

Architecture and non-linear expansion

While attention mechanisms model horizontal relationships across different sequence positions, they are linear weighted combinations of values. To introduce deep non-linear transformations and feature interactions, every Transformer layer incorporates a position-wise **Feed-Forward Network** (FFN) applied identically and independently to each token vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$:

$$\texttt{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\,\mathbf{W}_2 + \mathbf{b}_2, \tag{1.80}$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}}\times d_{\text{ff}}}$, $\mathbf{b}_1 \in \mathbb{R}^{d_{\text{ff}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}}\times d_{\text{model}}}$, and $\mathbf{b}_2 \in \mathbb{R}^{d_{\text{model}}}$.

In the original Transformer [**?**], the inner hidden dimension is expanded by a factor of 4:

$$d_{\text{ff}} = 4d_{\text{model}}. \tag{1.81}$$

For example, in the base model ($d_{\text{model}} = 512$), the intermediate dimension expands to $d_{\text{ff}} = 2048$ before contracting back to 512.

The FFN as a key-value associative memory

Recent theoretical investigations [**?**] demonstrate that Transformer FFN layers operate as **key-value associative memories** storing factual knowledge acquired during pretraining:

- The first layer $\mathbf{W}_1$ acts as a key detector: each intermediate neuron activates when a token's contextual state matches specific conceptual, semantic, or encyclopedic patterns.
- The non-linear activation (ReLU or GELU) enforces selective firing.
- The second layer $\mathbf{W}_2$ acts as a value provider, retrieving and injecting factual attributes and token updates into the residual stream.

The FFN layer accounts for approximately two-thirds of the total parameter count in a standard Transformer block.

1.5.3 Residual connections (skip connections)

In deep architectures stacking multiple layers ($N = 6$ to 96), backpropagating gradients through consecutive non-linear matrix transformations leads to rapid gradient decay or explosion. To resolve this, Vaswani et al. adopted **Residual Connections** [**?**] around every sublayer (Slide 75):

$$\mathbf{y} = \mathbf{x} + \mathcal{F}(\mathbf{x}), \tag{1.82}$$

where $\mathcal{F}(\mathbf{x})$ represents the Multi-Head Attention or Feed-Forward transformation.

Mathematical gradient analysis: the identity superhighway

To prove why residual connections prevent gradient extinction, we analyze the Jacobian of the residual update during backpropagation:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left(\mathbf{I} + \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}} \right) = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} + \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}}. \tag{1.83}$$

The presence of the identity matrix $\mathbf{I}$ establishes an unobstructed **gradient superhighway**. Even if the Jacobian of the learned sublayer vanishes entirely ($\frac{\partial \mathcal{F}}{\partial \mathbf{x}} \to \mathbf{0}$), the gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ propagates directly back to earlier layers without attenuation. This identity term enables the successful training of models spanning dozens or hundreds of stacked layers.

1.5.4 Layer Normalization

To ensure numerical stability and prevent activation values from exploding across deep networks, each sublayer is paired with **Layer Normalization** [**?**] (Slide 76).

Formulation and comparison with Batch Normalization

Unlike Batch Normalization (which computes statistics across the batch dimension for each feature, making it sensitive to batch size and variable sequence padding), Layer Normalization computes mean and variance statistics across the feature channels for each token vector independently:

Definition 1.1: *Layer Normalization*

For a token vector $\mathbf{x} = [x_1, x_2, \ldots, x_{d_{\text{model}}}]^\top \in \mathbb{R}^{d_{\text{model}}}$, Layer Normalization evaluates:

$$\mu = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} x_j, \quad \sigma^2 = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} (x_j - \mu)^2, \tag{1.84}$$

$$\texttt{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}, \tag{1.85}$$

where $\epsilon > 0$ is a small constant (e.g., 10^{-5}) added for numerical stability, and $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^{d_{\text{model}}}$ are learnable affine gain and bias parameters initialized to $\boldsymbol{\gamma} = \mathbf{1}$ and $\boldsymbol{\beta} = \mathbf{0}$.

Because statistics μ and σ^2 depend strictly on the individual token vector and are independent of batch size or neighboring tokens, Layer Normalization operates identically during training and evaluation, providing exceptional stability for variable-length natural language sequences.

Architectural placement: Post-LN vs. Pre-LN

- **Post-LN (Original Transformer [?])**: Normalization is applied *after* the residual sum:

$$\mathbf{y} = \texttt{LayerNorm}(\mathbf{x} + \mathcal{F}(\mathbf{x})). \tag{1.86}$$

  While mathematically elegant, Post-LN models suffer from vanishing gradients in early layers, requiring delicate learning rate warm-up schedules.

- **Pre-LN (Modern Standard [? ?])**: Normalization is applied to the input *before* the sublayer, preserving an unbroken identity shortcut:

$$\mathbf{y} = \mathbf{x} + \mathcal{F}\left(\texttt{LayerNorm}(\mathbf{x})\right). \tag{1.87}$$

  Pre-LN significantly stabilizes gradient flow, enabling deeper stacking without warm-up instability.

1.5.5 End-to-end Transformer assembly and the translation journey

Full architectural assembly

Assembling these core blocks produces the canonical Encoder-Decoder Transformer architecture (Figure 1.7). The Encoder consists of a stack of $N = 6$ identical layers, each containing a Bidirectional Multi-Head Self-Attention sublayer and an FFN sublayer. The Decoder consists of a stack of $N = 6$ identical layers, each containing a Causal Masked Multi-Head Self-Attention sublayer, an Encoder-Decoder Multi-Head Cross-Attention sublayer, and an FFN sublayer.

Step-by-step translation journey: English to Italian

To understand how information traverses the entire architecture, consider translating the English sentence *"The bank of the river"* into Italian: *"La riva del fiume"*.
Phase 1: Source Sequence Ingestion in the Encoder.

1. **Tokenization**: The source text decomposes into $S = 5$ tokens:

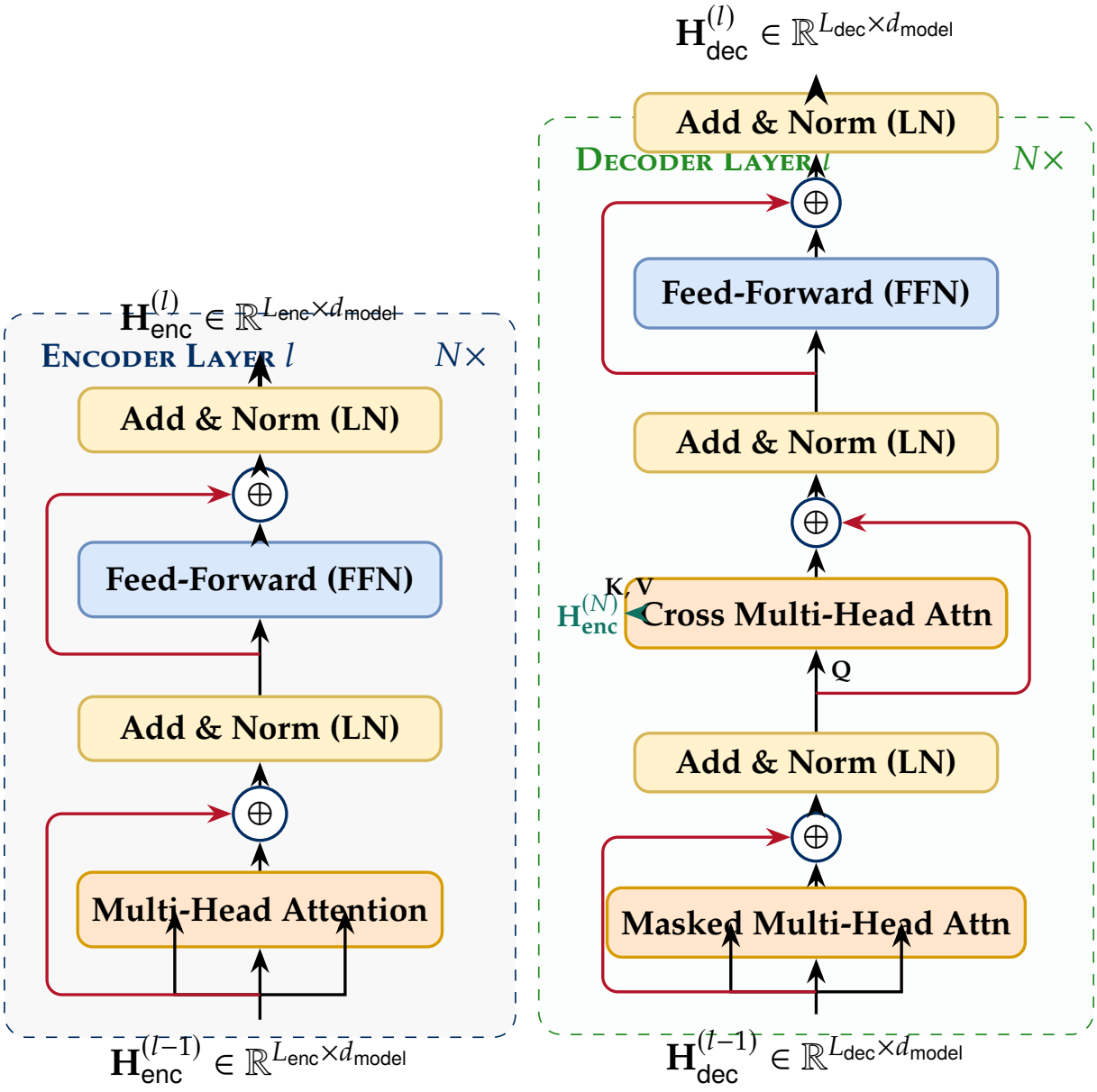
$$X_{\text{src}} = [\texttt{"The"}, \texttt{"bank"}, \texttt{"of"}, \texttt{"the"}, \texttt{"river"}]. \tag{1.88}$$

2. **Embedding and Positional Injection**: Each token indexes the embedding matrix $\mathbf{W}_{\text{emb}} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$, and sinusoidal positional encodings are additively combined:

$$\mathbf{H}_{\text{enc}}^{(0)} = \mathbf{E}_{\text{tok}} + \mathbf{PE} \in \mathbb{R}^{5 \times d_{\text{model}}}. \tag{1.89}$$

3. **Encoder Layers (1 to $N = 6$)**:
   - *Bidirectional Self-Attention*: Every token attends to all 5 tokens. For example, token `"bank"` computes high attention with `"river"`, absorbing geographical context.
   - *Add & Norm*: Residual skip connection adds the input $\mathbf{H}_{\text{enc}}^{(l-1)}$, followed by LayerNorm.
   - *FFN & Add & Norm*: Non-linear expansion to $4d_{\text{model}}$ and contraction back to d_{model}, followed by residual addition and LayerNorm.

4. **Terminal Encoder Memory**: After 6 layers, the encoder outputs the continuous memory tensor $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{5 \times d_{\text{model}}}$. Unlike RNNs, this is *not* a single summary vector, but an uncompressed sequence of 5 deeply contextualized vectors.



(a) Encoder layer stack ($N\times$)

(b) Decoder layer stack ($N\times$)

Figure 1.7: Complete structural assembly of the Transformer architecture [**?**].

Phase 2: Autoregressive Target Decoding (Inference).
At test time, the target sequence is generated autoregressively, token by token (Slides 8–9):

1. **Step 1: Generating the first target token ("La")**:
   - The decoder input is the single special token: ["<BOS>"].
   - *Masked Self-Attention*: "<BOS>" attends only to itself.
   - *Cross-Attention*: The query from "<BOS>" queries the 5 keys from encoder memory $\mathbf{H}_{\text{enc}}$, attending to "The" and the global topic "river".
   - *FFN and Output Projection*: The representation passes through the FFN, projects via $\mathbf{W}_{\text{head}} \in \mathbb{R}^{|\mathcal{V}_{\text{target}}| \times d_{\text{model}}}$ into vocabulary logits, and softmax samples token 1: "La".
2. **Step 2: Generating the second target token ("riva")**:
   - The decoder input expands to: ["<BOS>", "La"].
   - *Masked Self-Attention*: Token "La" attends to "<BOS>" and itself, but is prevented from seeing future positions.
   - *Cross-Attention*: Token "La" queries the encoder keys. Because the model must translate "bank", and the encoder memory for "bank" is contextualized by "river", cross-attention aligns "La" with "bank", extracting the value representation for geological riverbank.
   - The classification head projects the state and emits "riva".
3. **Step 3 to 5: Sequential completion**: The decoder continues iteratively:

$$\text{["<BOS>","La","riva"]} \implies \text{"del"}, \tag{1.90}$$
$$\text{["<BOS>","La","riva","del"]} \implies \text{"fiume"}, \tag{1.91}$$
$$\text{["<BOS>","La","riva","del","fiume"]} \implies \text{"<EOS>"}. \tag{1.92}$$

   When "<EOS>" is emitted, the generation process halts.

Parallel training vs. autoregressive inference

As highlighted in Slide 80, a profound architectural strength of the Transformer is the duality between training and inference:

- **Training Phase (Teacher Forcing)**: The full target sentence is known in advance. By feeding the entire ground-truth sequence shifted right into the decoder simultaneously and applying the causal mask $\mathbf{M}$, all T target token probabilities $P(y_t \mid y_{<t})$ and cross-entropy losses are computed concurrently in a single forward pass ($O(1)$ sequential steps). Training across billions of tokens scales with massive GPU parallelism.
- **Inference Phase**: Autoregressive generation cannot know the future. Tokens must be generated sequentially in T forward steps ($O(T)$ sequential operations), each step appending the newly sampled token to the prompt context.

1.5.6 Computational complexity and the quadratic attention barrier

Table 1.5 compares the asymptotic complexity of Self-Attention, Recurrent, and Convolutional layers across sequence length L and representation dimension d.

1. **Parallel Execution**: Computing the attention matrix $\mathbf{QK}^\top$ and weighted sum $\mathbf{AV}$ involves dense matrix multiplications that execute in a single parallel step ($O(1)$ sequential operations) on GPU tensor cores, shattering the recurrent execution wall.

Table 1.5: Asymptotic complexity across layer paradigms per layer [**?**].

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(L^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent (RNN)	$O(L \cdot d^2)$	$O(L)$	$O(L)$
Convolutional	$O(k \cdot L \cdot d^2)$	$O(1)$	$O(\log_k(L))$

2. **Constant Path Length**: The distance between any arbitrary pair of tokens (i, j) is strictly $O(1)$, eliminating long-range gradient decay.

3. **The Quadratic Memory Barrier**: Computing and storing the attention matrix $\mathbf{A} \in \mathbb{R}^{L \times L}$ incurs quadratic time and memory complexity $O(L^2 \cdot d)$. While optimal for moderate sequence lengths ($L \leq 2048$), expanding context lengths to tens of thousands of tokens encounters the quadratic memory wall, motivating modern sparse, flash, and linear attention architectures.